Week 7: 8086 Registers & FLAGS, Memory Segmentation & Addresses

Midterm Exam Preparation
Week 7

According to your syllabus, Week 7 contains two major parts:

1. 8086 Registers & FLAGS

- AX, BX, CX, DX
- SP, BP, SI, DI, IP
- FLAGS register
- Important status and control flags

2. Memory Segmentation & Addresses

- Segment registers: CS, DS, SS, ES
- Offset
- Logical address
- Physical address
- Basic address calculation

This week is particularly important because it includes both theoretical questions and numerical problems. Let's understand everything step by step.

Part 1: 8086 Registers

1. What are Registers?

Definition: Registers are small, high-speed storage locations inside the CPU that temporarily store data, instructions, memory addresses, and intermediate results.

The 8086 microprocessor has several 16-bit registers.

Why are registers important?

- They store data temporarily.
- They hold memory addresses.
- They help perform arithmetic and logical operations.
- They store instruction-related information.
- They improve processing speed because accessing registers is faster than accessing main memory.

Types of 8086 Registers

The 8086 registers can be divided into these categories:

1. General-purpose registers
2. Pointer and index registers
3. Instruction Pointer
4. Segment registers
5. FLAGS register

2. General-Purpose Registers

The 8086 has four main general-purpose registers:

- AX – Accumulator Register
- BX – Base Register
- CX – Count Register
- DX – Data Register

All four are 16-bit registers.

[8086 Microprocessor Architecture: 16-bit microprocessor | PPT
slideshare.net]

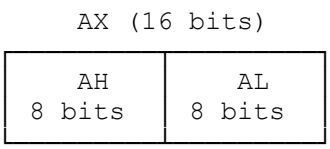
A. AX – Accumulator Register

Definition: AX is the accumulator register, mainly used in arithmetic, logical, and data-processing operations.

It is commonly used in multiplication, division, and input/output operations.

AX is divided into two 8-bit parts:

- AH = Higher 8 bits
- AL = Lower 8 bits



Example:

```
MOV AX, 1234H
```

Now:

```
AX = 1234H
AH = 12H
AL = 34H
```

Remember: AX is mainly used as an accumulator for arithmetic and data operations.

B. BX – Base Register

Definition: BX is the base register, commonly used to store base addresses for memory access.

It can help access data stored in memory.

BX is divided into:

- BH = Higher 8 bits
- BL = Lower 8 bits

Example:

```
MOV BX, 2000H
```

Now:

```
BX = 2000H
```

BX can be used as a base register in memory addressing.

Remember: BX = Base Register.

C. CX – Count Register

Definition: CX is the count register, commonly used to store loop counters and repetition counts.

It is frequently used with loop and string instructions.

CX is divided into:

- CH = Higher 8 bits
- CL = Lower 8 bits

Example:

```
MOV CX, 05H
```

This means the count register contains 5.

```
LOOP START
```

The LOOP instruction decreases CX by 1 and repeats the loop while CX is not zero.

Remember: CX = Count Register.

D. DX – Data Register

Definition: DX is the data register, commonly used in multiplication, division, and input/output operations.

It can also hold the upper part of certain arithmetic results.

DX is divided into:

- DH = Higher 8 bits
- DL = Lower 8 bits

Example:

```
MOV DX, 1234H
```

Now:

```
DX = 1234H
DH = 12H
DL = 34H
```

Remember: DX = Data Register.

Comparison of General-Purpose Registers

Register	Full Name	Main Purpose
AX	Accumulator	Arithmetic and data operations
BX	Base	Base addressing
CX	Count	Loops and repetition
DX	Data	Multiplication, division, I/O

Important exam point: AX, BX, CX, and DX are 16-bit registers, and each can be divided into two separate 8-bit registers.

3. Pointer and Index Registers

The 8086 has four important pointer and index registers:

1. SP – Stack Pointer

2. BP – Base Pointer
3. SI – Source Index
4. DI – Destination Index

All are 16-bit registers.

Register Names | Flyxiv Blog
junyeopn.github.io

A. SP – Stack Pointer

Definition: The Stack Pointer (SP) stores the offset address of the top of the stack.

The stack is a memory area used for temporarily storing data, return addresses, and other information.

Example:

```
MOV SP, 1000H
```

Now:

```
SP = 1000H
```

It points to the top of the stack within the stack segment.

Remember: SP = Stack Pointer.

B. BP – Base Pointer

Definition: The Base Pointer (BP) is used to access data stored in the stack segment.

It is especially useful for accessing function parameters and local data in stack-based operations.

Example:

```
MOV BP, 2000H
```

Now:

```
BP = 2000H
```

Remember: BP = Base Pointer.

C. SI – Source Index

Definition: The Source Index (SI) stores the offset address of source data, particularly in string and indexed memory operations.

It is commonly associated with the Data Segment (DS).

Example:

```
MOV SI, 1000H
```

Now:

```
SI = 1000H
```

Remember: SI = Source Index.

D. DI – Destination Index

Definition: The Destination Index (DI) stores the offset address of destination data, particularly in string and indexed memory operations.

It is commonly associated with the Extra Segment (ES) in string operations.

Example:

```
MOV DI, 2000H
```

Now:

```
DI = 2000H
```

Remember: DI = Destination Index.

Comparison

Register	Full Name	Main Purpose
SP	Stack Pointer	Points to top of stack
BP	Base Pointer	Accesses stack data
SI	Source Index	Points to source data
DI	Destination Index	Points to destination data

4. Instruction Pointer (IP)

Definition: The Instruction Pointer (IP) is a 16-bit register that stores the offset address of the next instruction to be executed within the Code Segment.

It works with the CS register.

Example:

```
CS = 2000H
IP = 0100H
```

This means that the next instruction is located at logical address:

```
2000H:0100H
```

The BIU uses CS and IP to calculate the physical address of the instruction.

Important points:

- IP is a 16-bit register.
- It holds an instruction offset.
- It works with CS.
- It helps maintain the sequence of instruction execution.

Remember: IP = Address of the next instruction's location within the code segment.

Part 2: FLAGS Register

5. What is the FLAGS Register?

Definition: The FLAGS register is a 16-bit register in the 8086 that stores status and control information about CPU operations.

It contains individual bits called flags.

These flags indicate the results of arithmetic and logical operations and control certain processor operations.

Flag Register in 8086 microprocessor
microtrickslearning.blogspot.com

The 8086 has nine commonly used flags:

- Six status flags
- Three control flags

6. Status Flags

Definition: Status flags indicate the result or condition of an arithmetic or logical operation.

There are six important status flags.

A. Carry Flag (CF)

Definition: The Carry Flag is set to 1 when an arithmetic operation generates a carry out of the most significant bit or a borrow in subtraction.

Example:

```
  1111
+ 0001
------
 10000
```

If we consider a 4-bit result:

```
Result = 0000
Carry = 1
```

Therefore:

```
CF = 1
```

Remember: CF indicates carry or borrow.

B. Zero Flag (ZF)

Definition: The Zero Flag is set to 1 when the result of an operation is zero.

Example:

```
5 - 5 = 0
```

Therefore:

```
ZF = 1
```

If the result is non-zero:

```
5 + 3 = 8
ZF = 0
```

Remember: ZF = 1 means result is zero.

C. Sign Flag (SF)

Definition: The Sign Flag indicates the sign of a result by copying its most significant bit.

For an 8-bit result:

- MSB = 0 → Positive or non-negative sign
- MSB = 1 → Negative sign in signed representation

Example:

```
Result = 10000000
```

The most significant bit is 1.

Therefore:

```
SF = 1
```

Remember: SF indicates the sign of the result.

D. Overflow Flag (OF)

Definition: The Overflow Flag is set when a signed arithmetic operation produces a result that is outside the range that can be represented.

Example using 8-bit signed numbers:

```
  01111111  (+127)
+ 00000001  (+1)
-----------
  10000000  (-128 if interpreted as signed)
```

The actual result should be +128, but an 8-bit signed number cannot represent +128.

Therefore:

```
OF = 1
```

Remember: OF indicates signed arithmetic overflow.

E. Parity Flag (PF)

Definition: The Parity Flag indicates whether the lower 8 bits of the result contain an even or odd number of 1s.

In 8086:

- Even number of 1s → PF = 1
- Odd number of 1s → PF = 0

Example:

```
Result = 00000101
```

There are two 1s.

Therefore:

```
PF = 1
```

Another example:

```
Result = 00000111
```

There are three 1s.

Therefore:

```
PF = 0
```

Remember: Even parity = 1; odd parity = 0.

F. Auxiliary Carry Flag (AF)

Definition: The Auxiliary Carry Flag is set when a carry occurs from bit 3 to bit 4 during an arithmetic operation.

It is mainly used in Binary-Coded Decimal (BCD) arithmetic.

Example:

```
00001111
+00000001
---------
00010000
```

A carry occurs from the lower 4 bits into the upper 4 bits.

Therefore:

```
AF = 1
```

Remember: AF indicates carry from bit 3 to bit 4.

7. Control Flags

Definition: Control flags control certain operations of the processor.

The 8086 has three main control flags.

A. Trap Flag (TF)

Definition: The Trap Flag enables single-step execution for debugging.

When TF = 1, the processor generates a single-step interrupt after each instruction.

This helps programmers debug programs one instruction at a time.

```
TF = 1 → Single-step mode enabled
TF = 0 → Single-step mode disabled
```

B. Interrupt Flag (IF)

Definition: The Interrupt Flag controls whether maskable hardware interrupts are enabled or disabled.

```
IF = 1 → Maskable interrupts enabled
IF = 0 → Maskable interrupts disabled
```

Note: IF does not disable non-maskable interrupts (NMI).

C. Direction Flag (DF)

Definition: The Direction Flag controls the direction in which string operations process data.

```
DF = 0 → Forward direction
DF = 1 → Backward direction
```

For example, during string operations:

- DF = 0: SI and DI increase.
- DF = 1: SI and DI decrease.

8. Complete FLAGS Register Summary

Important for Short Questions and MCQs

Flag	Full Name	Purpose
CF	Carry Flag	Indicates carry or borrow
ZF	Zero Flag	Indicates zero result
SF	Sign Flag	Indicates sign of result
OF	Overflow Flag	Indicates signed overflow
PF	Parity Flag	Indicates even or odd parity
AF	Auxiliary Carry Flag	Carry from bit 3 to bit 4

TF	Trap Flag	Enables single-step debugging
IF	Interrupt Flag	Controls maskable interrupts
DF	Direction Flag	Controls string-processing direction

Easy way to remember

Status flags:

CF, PF, AF, ZF, SF, OF

Control flags:

TF, IF, DF

Part 3: Memory Segmentation & Addresses

9. What is Memory Segmentation?

Definition: Memory segmentation is a memory-management technique in which the memory of a program is divided into logical sections called segments.

The 8086 uses memory segmentation to organize its 1 MB memory space.

Why is segmentation used?

- To organize program instructions and data.
- To separate code, data, and stack.
- To support efficient memory addressing.
- To allow different memory segments to be accessed using 16-bit segment and offset values.

Four Main Segments in 8086

1. Code Segment (CS)
2. Data Segment (DS)
3. Stack Segment (SS)
4. Extra Segment (ES)

Computer Systems Principle [1] - Registers, Address Modes | Vines' Log
vinesmsuic.github.io

10. Segment Registers

Definition: Segment registers are 16-bit registers that store the starting segment addresses used to access different memory areas.

The 8086 has four segment registers.

A. CS – Code Segment

Definition: The Code Segment register stores the starting address of the memory segment that contains program instructions.

It works with IP.

Example:

```
CS = 2000H
IP = 0100H
```

Together they identify the location of the next instruction.

Remember: CS = Stores the code segment address.

B. DS – Data Segment

Definition: The Data Segment register stores the starting address of the segment that contains program data.

It is commonly used when accessing variables and other data.

Example:

```
DS = 3000H
```

The processor can use DS with an offset to access data.

Remember: DS = Stores the data segment address.

C. SS – Stack Segment

Definition: The Stack Segment register stores the starting address of the memory segment used for the stack.

It works with SP or BP to access stack data.

Example:

```
SS = 4000H
SP = 0100H
```

Together, SS and SP identify the current top of the stack.

Remember: SS = Stores the stack segment address.

D. ES – Extra Segment

Definition: The Extra Segment register stores the starting address of an additional data segment.

It is commonly used as the destination segment in string operations.

Example:

```
ES = 5000H
```

Remember: ES = Additional data segment.

Comparison of Segment Registers

Register	Full Name	Purpose
CS	Code Segment	Stores program instructions
DS	Data Segment	Stores program data
SS	Stack Segment	Stores stack data
ES	Extra Segment	Provides an additional data segment

Important: Each segment register is 16 bits wide.

11. What is an Offset Address?

Definition: An offset is a 16-bit value that specifies the location of data or an instruction relative to the beginning of a memory segment.

It identifies a particular location within a segment.

Example:

```
DS = 2000H
Offset = 0100H
```

Here:

- DS identifies the data segment.
- 0100H identifies a location inside that segment.

The segment and offset are combined to calculate the physical address.

Remember: Segment tells us the starting location, and offset tells us the distance from that starting location.

12. What is a Logical Address?

Definition: A logical address in 8086 is an address represented by two 16-bit values: a segment address and an offset address.

It is normally written in this format:

$$\text{Segment}:\text{Offset}$$

Example:

```
2000H:0100H
```

Here:

- Segment = 2000H
- Offset = 0100H

A logical address is converted into a physical address to access the actual memory location.

Important: A logical address is not the same as a physical address.

13. What is a Physical Address?

Most Important Numerical Topic

Definition: A physical address is the actual 20-bit address of a memory location in the computer's main memory.

The 8086 generates a physical address by combining a segment address and an offset address.

Physical Address Formula

$$\boxed{\text{Physical Address}= (\text{Segment}\times 10H)+\text{Offset}}$$

Why multiply by 10H?

Because the 8086 shifts the segment value left by 4 bits, which is equivalent to multiplying it by 16 (decimal) or 10H (hexadecimal).

In simple terms:

```
Segment address × 10H
+
Offset address
=
Physical address
```

Example 1: Calculate Physical Address

Given:

```
Segment = 2000H
Offset  = 0010H
```

Step 1: Apply the formula

\[PA=(2000H\times 10H)+0010H \]

Step 2: Multiply segment by 10H

\[2000H\times10H=20000H \]

Step 3: Add offset

\[20000H+0010H=20010H \]
Final Answer
\[\boxed{20010H} \]

Example 2: Calculate Physical Address

Given:

```
Segment = 3000H
Offset  = 0020H
```

Step 1:

\[3000H\times10H=30000H \]

Step 2:

\[30000H+0020H=30020H \]

Answer:

\[\boxed{30020H} \]

Example 3: Calculate Physical Address

Given:

```
Segment = 1234H
Offset  = 5678H
```

Step 1:

\[1234H\times10H=12340H \]

Step 2:

\[12340H+5678H=179B8H \]

Answer:

\[\boxed{179B8H} \]

Example 4: Calculate Physical Address

Given:

```
Segment = 2500H
Offset  = 1000H
```

Step 1:

\[2500H\times10H=25000H \]

Step 2:

\[25000H+1000H=26000H \]

Answer:

\[\boxed{26000H} \]

14. How to Solve Physical Address Numericals Easily

Always follow these three steps:

Step 1: Write the segment and offset

```
Segment = 2000H
Offset  = 0100H
```

Step 2: Multiply the segment by 10H

```
2000H × 10H = 20000H
```

Step 3: Add the offset

```
20000H
+ 0100H
-------
20100H
```

20100H

Physical address

Important numerical tip: Always perform the addition in hexadecimal, not decimal. Remember that A = 10, B = 11, C = 12, D = 13, E = 14, and F = 15 in hexadecimal.

15. Logical Address vs Physical Address

Logical Address	Physical Address
Contains segment and offset	Actual memory address
Represented by two 16-bit values	20-bit address in 8086
Written as Segment:Offset	Written as a single hexadecimal address
Example: 2000H:0010H	Example: 20010H

Example

Given:

```
Logical Address = 2000H:0010H
```

Calculation:

\[PA=(2000H\times10H)+0010H \]
\[PA=20000H+0010H \]
\[\boxed{PA=20010H} \]

16. Important Concept: Different Logical Addresses Can Have the Same Physical Address

This is an important concept that may appear in conceptual or numerical questions.

Because a segment is shifted by only 4 bits, different segment-offset combinations can point to the same physical memory location.

Example 1:

```
2000H:0010H
```

Physical address:

\[20000H+0010H=20010H \]

Example 2:

```
2001H:0000H
```

Physical address:

\[20010H+0000H=20010H \]

Both logical addresses produce the same physical address.

\[\boxed{2000H:0010H=2001H:0000H} \]

in terms of physical memory location.

Why? Because different segment-offset combinations can identify overlapping memory locations.

17. Relationship Between Registers and Memory Addresses

This is important for understanding how the 8086 uses registers.

Register Combination	Purpose
CS:IP	Address of next instruction
DS:Offset	Accesses data
SS:SP	Accesses top of stack
SS:BP	Accesses data in stack
DS:SI	Commonly accesses source data
ES:DI	Commonly accesses destination data

For example:

```
CS = 2000H
IP = 0100H
```

The physical address of the next instruction is:

\[(2000H\times10H)+0100H \]
\[\boxed{20100H} \]

Similarly, if:

```
DS = 3000H
SI = 0020H
```

The physical address of the source data is:

\[30000H+0020H \]
\[\boxed{30020H} \]

Important Exam Questions

Long Questions

Prepare these questions thoroughly:

1. Explain the different types of registers in the 8086 microprocessor.
2. Explain AX, BX, CX, and DX registers with their functions.
3. Explain SP, BP, SI, and DI registers.
4. What is the Instruction Pointer? Explain its role in instruction execution.
5. What is the FLAGS register? Explain all status and control flags.
6. Differentiate between status flags and control flags.
7. Explain memory segmentation in the 8086 microprocessor.
8. Explain CS, DS, SS, and ES segment registers.
9. Differentiate between logical and physical addresses.
10. Explain the physical address calculation formula with examples.

Short Questions and Answers

Q1. How many bits are the 8086 general-purpose registers?

They are 16-bit registers.

Q2. What are the four general-purpose registers?

AX, BX, CX, and DX.

Q3. What is the purpose of AX?

AX is the accumulator register, commonly used in arithmetic and data operations.

Q4. What is the purpose of CX?

CX is used as a count register for loops and repetition operations.

Q5. What is SP?

SP is the Stack Pointer, which stores the offset address of the top of the stack.

Q6. What is SI?

SI is the Source Index register, commonly used to access source data.

Q7. What is IP?

IP stores the offset address of the next instruction within the code segment.

Q8. What is the FLAGS register?

It is a 16-bit register that stores status and control information.

Q9. How many commonly used flags are there in 8086?

There are nine commonly used flags: six status flags and three control flags.

Q10. What does CF indicate?

CF indicates carry or borrow in arithmetic operations.

Q11. What does ZF indicate?

ZF indicates whether the result of an operation is zero.

Q12. What does OF indicate?

OF indicates signed arithmetic overflow.

Q13. What is memory segmentation?

It is the technique of dividing memory into logical sections called segments.

Q14. Name the four segment registers.

CS, DS, SS, and ES.

Q15. What is an offset?

An offset is a 16-bit value that identifies a location relative to the beginning of a segment.

Q16. What is a logical address?

A logical address consists of a segment address and an offset address.

Q17. What is a physical address?

It is the actual 20-bit memory address generated by the 8086.

Q18. What is the formula for physical address calculation?

\[PA=(Segment\times10H)+Offset \]

Numerical Practice

Try these yourself before checking the solutions.

Solve these five questions

Q1. Segment = 1000H, Offset = 0020H. Find the physical address.

Show solution

Q2. Segment = 2500H, Offset = 0100H. Find the physical address.

Show solution

Q3. Segment = 1234H, Offset = 0010H. Find the physical address.

Show solution

Q4. Segment = 4000H, Offset = 0200H. Find the physical address.

Show solution

Q5. Segment = 3000H, Offset = 1234H. Find the physical address.

Show solution

Quick Revision: What You Must Memorize

Topic	Key Point
AX	Accumulator
BX	Base
CX	Count
DX	Data
SP	Stack Pointer
BP	Base Pointer
SI	Source Index
DI	Destination Index
IP	Next instruction offset
FLAGS	Six status + three control flags
CS	Code Segment
DS	Data Segment
SS	Stack Segment
ES	Extra Segment
Logical Address	Segment:Offset
Physical Address	Actual 20-bit address

Most Important Formulas

$$\boxed{PA=(Segment\times10H)+Offset}$$
$$\boxed{20\text{-bit address bus}=1\text{ MB memory}}$$
$$\boxed{CS:IP=\text{Next instruction location}}$$

My recommendation for your exam

Give extra practice to these four areas:

1. FLAGS: Learn the purpose of all nine flags, especially CF, ZF, SF, OF, IF, and DF.
2. 8086 registers: Memorize each register's full name and function.
3. Segment registers: Understand the difference between CS, DS, SS, and ES.
4. Physical address numericals: Practice at least 5–10 questions using the segment × 10H + offset formula.

These are the core concepts of Week 7, and you should be able to solve the physical-address questions independently before moving to Week 8.